EXPERIMENT 14

Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Name: *Kanishka S* | ***Register Number:*** *240701227*

Aim

To explore how pro-code capabilities can extend a low-code Zoho Creator application using Zoho Catalyst / custom functions, by creating and executing a custom Deluge function in the Smart Campus Sustainability & Carbon Tracker application.

Introduction

Low-code platforms allow applications to be developed using visual components such as forms, reports and workflows, with little hand-written code. This makes it quick to build standard business applications such as a campus sustainability tracker, where staff record energy, water and waste data and the sustainability team follows the results.

When the standard low-code functionality is insufficient for a particular requirement, pro-code extensions allow developers to introduce programmable logic into the application. Zoho Creator supports this through Deluge, its scripting language, and through custom functions, which are reusable units of Deluge code that can be called within an application when required. Zoho Catalyst, Zoho's serverless development platform, represents a further pro-code option for requirements that go beyond what can be scripted inside the application.

In this experiment, a custom function named getTotalEnergy was created in Zoho Creator and executed to observe how a pro-code extension is created and run. The function uses the Deluge language and the Default namespace, takes no arguments and has the return type float. It returns the total electricity consumed, calculated from the Energy_Consumption form.

Pro-Code Extension in Zoho Creator

Extending a low-code application with custom functions means that the visual structure of the application (forms, reports and workflows) is retained, while specific logic is written as code in a function and used where required. The main concepts are as follows:

- **Custom functions:** named, reusable functions created in the Functions section of the Workflow area. A function has a name, a namespace, a return type and optional arguments.
- **Deluge scripting:** the scripting language of Zoho Creator, in which the logic of a custom function is written. The Deluge editor provides a list of ready-made statements such as conditions, data-access operations, set variable, call function and return.
- **Reusable application logic:** a function is written once and can be called from within the application whenever its logic is needed, which avoids repeating the same code. For example, the same function can be used by the AI assistant and by a dashboard report.
- **Custom business logic:** application-specific rules that are not covered by standard configuration, such as emission calculations or consumption thresholds, can be implemented in a function.
- **Data processing:** Deluge provides operations to add, fetch, aggregate, update and delete records, which allow energy, water and waste records to be processed through code.

- **Automation and extensibility:** functions allow automation to be extended beyond standard options, so that the application can be adapted to specific requirements.

Figure 1 illustrates the concept. A user works with the Smart Campus application, which has components such as forms, workflows, reports and custom functions. The pro-code extension (Deluge scripting, custom functions, business logic, data processing and automation) is represented by the custom function getTotalEnergy(), which takes no arguments, runs Deluge logic and returns the total energy as a float.

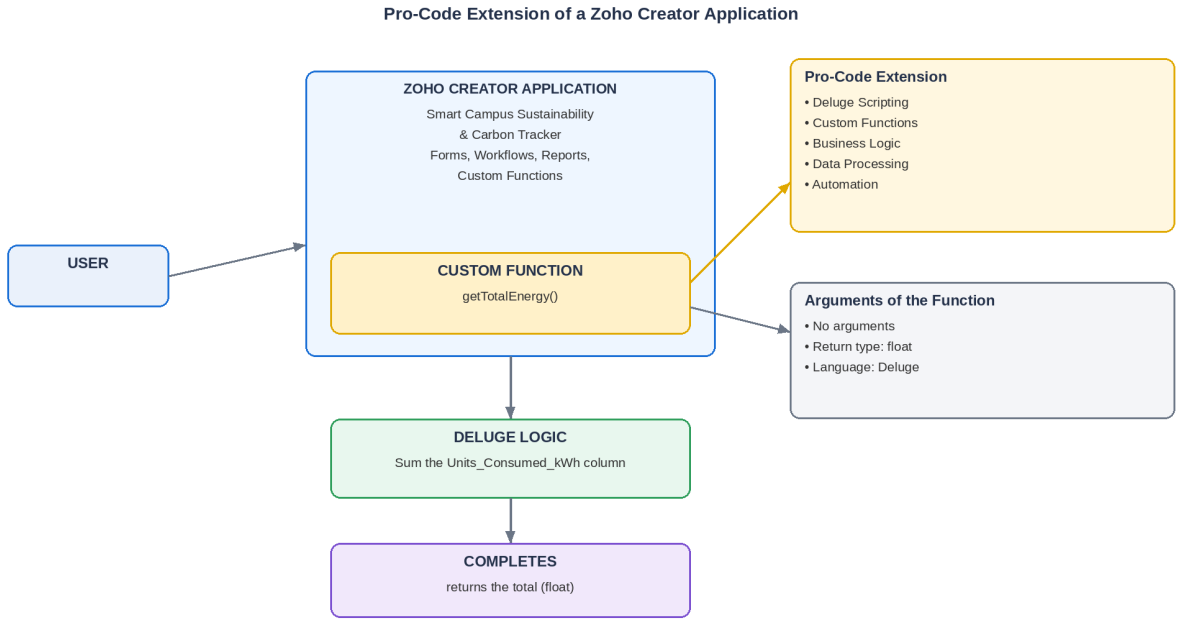


Figure 1: Pro-Code Extension of a Zoho Creator Application (Supporting Diagram)

The diagram is a conceptual illustration. In this experiment, only the creation and execution of the custom function were carried out; the function was not attached to a workflow or connected to any other component.

Procedure

1 Open the application in Zoho Creator in Edit mode.

2 Navigate to the Workflow section (Figure 2).

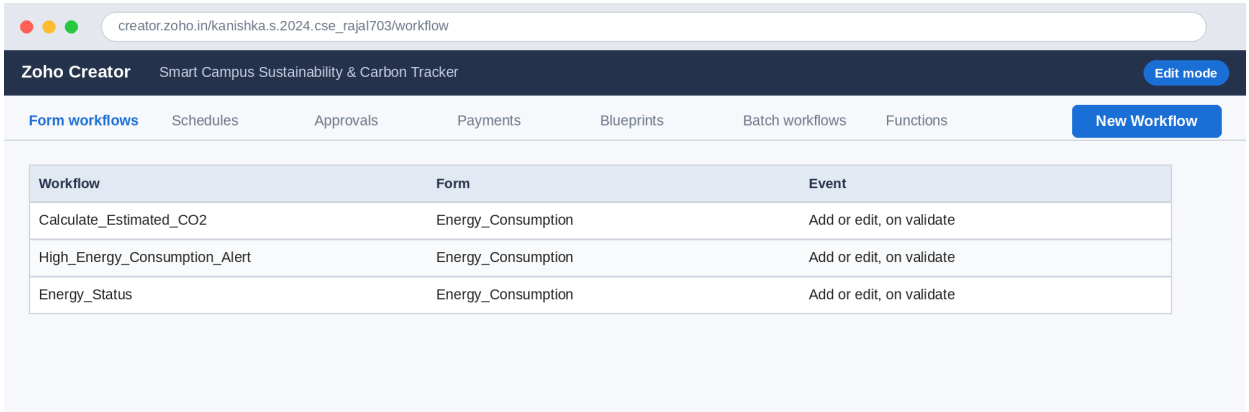


Figure 2: Workflow Section of the Application (illustration)

3 Open the Functions section from the tabs of the Workflow area (next to Batch workflows) and select the option to create a new function.

4 The “Execute a function when called” form is displayed, with the fields Name the function, Select the language, Select namespace, Select return type and Specify arguments (Figure 3).

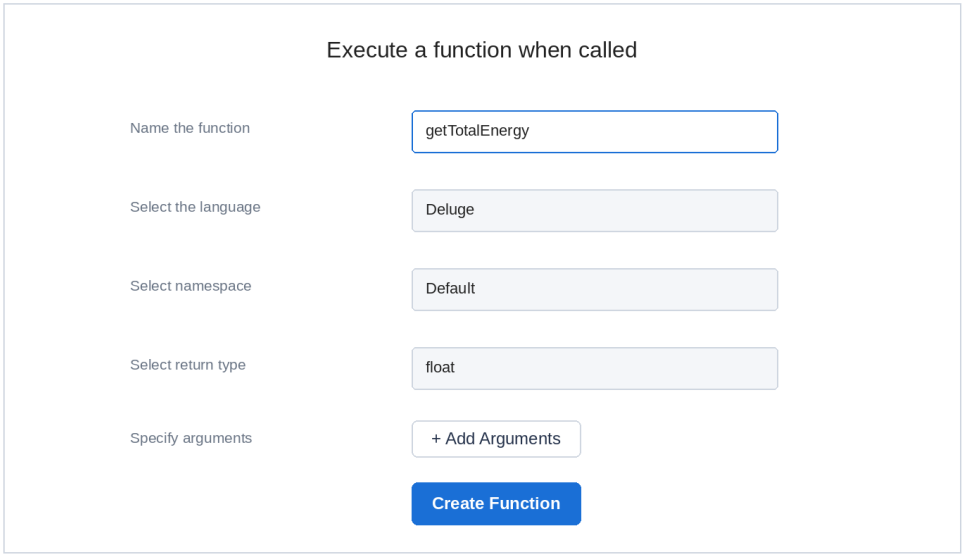


Figure 3: Create Function Form – Name, Language, Namespace, Return Type and Arguments (illustration)

5 Enter the function name getTotalEnergy.

6 Select Deluge as the programming language.

7 Use the Default namespace.

8 Set the return type to float.

9 No arguments are added for this function. Select Create Function.

10 The Deluge editor opens with the function declaration float getTotalEnergy() generated from the selected options (Figure 4). The editor also lists ready-made statements such as Condition, Data Access and Miscellaneous on the left. Write the function body as shown below.

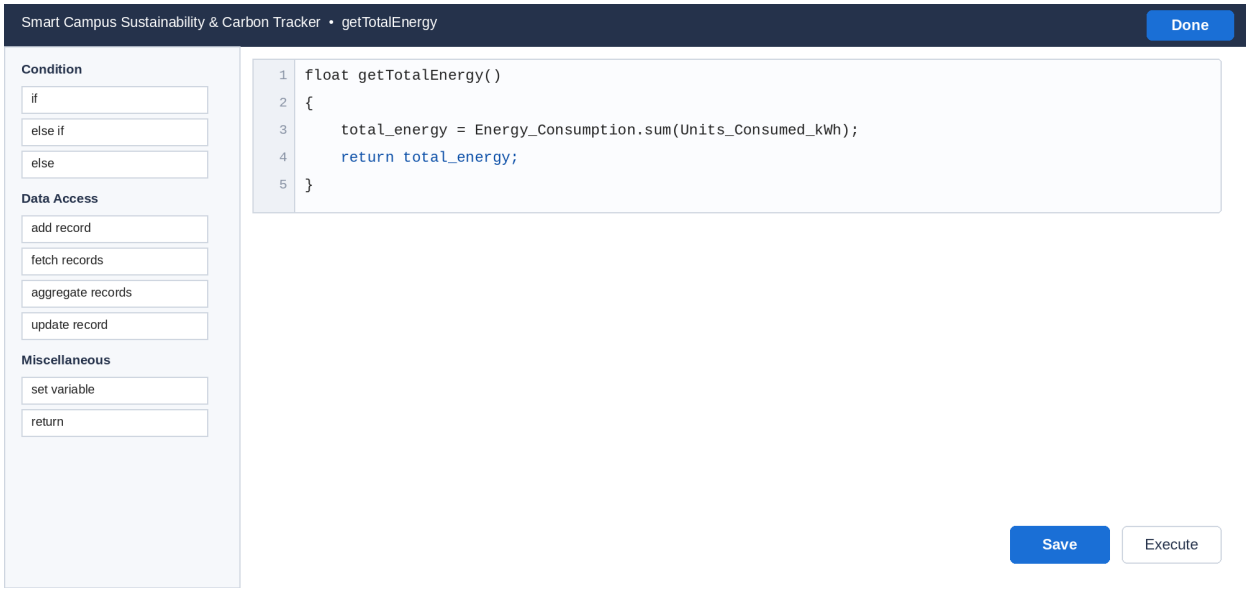


Figure 4: Deluge Editor – Function getTotalEnergy() (illustration)

11 Save the custom function using Save.

12 Execute the function using Execute. As the function has no arguments, no input values are requested, and the Execute window reports the result (Figure 5).

13 Verify that the Execute window reports Executed Successfully (Figure 5). The saved function is then listed in the Functions section (Figure 6).

Custom Function Implementation

The Deluge code of the custom function is shown below. It sums the Units_Consumed_kWh column of the Energy_Consumption form with a single aggregate operation and returns the total:

```
float getTotalEnergy()
{
    total_energy = Energy_Consumption.sum(Units_Consumed_kWh);
    return total_energy;
}
```

The function is declared with the return type float, so it returns a value, and it takes no arguments. Using the built-in aggregate sum avoids fetching and looping through every record. The function details are summarised below.

Property	Value
Function Name	getTotalEnergy
Language	Deluge
Namespace	Default
Return Type	float
Arguments	None
Syntax	getTotalEnergy()
Execution Result	Executed Successfully (returned 2450.0 for the three test records of Experiment 6)

Function Execution

The custom function was executed in Zoho Creator using the Execute option of the Deluge editor. Because the function has no arguments, it ran without asking for any input. The Execute window displayed the result Executed Successfully along with the returned total of 2450.0 kWh (500 + 950 + 1000 kWh from the test records), confirming that the function ran without errors (Figure 5).

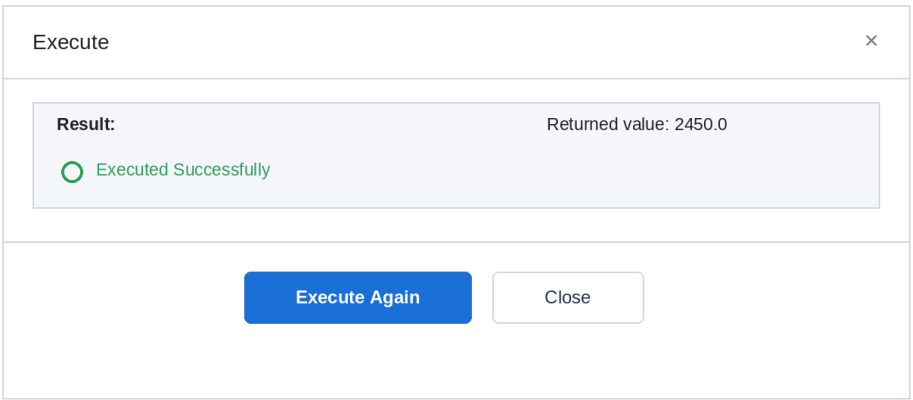


Figure 5: Execution of getTotalEnergy() Custom Function (illustration)

Saved Custom Function

After saving, the function appears in the Functions section of the Workflow area. It is listed under the Default namespace with the function name getTotalEnergy, the language Deluge, the return type float and the syntax getTotalEnergy() (Figure 6).

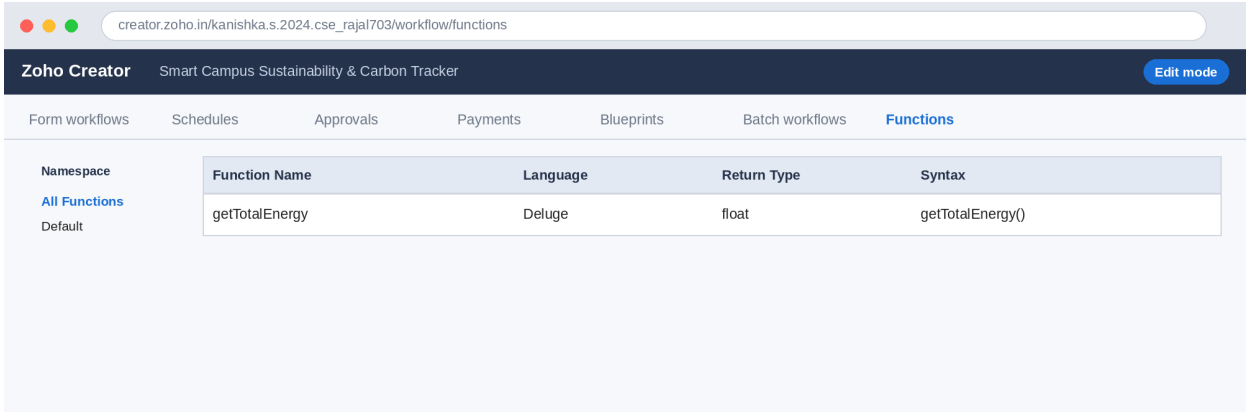


Figure 6: Saved getTotalEnergy Custom Function (illustration)

Execution Flow

Figure 7 shows the execution flow of the custom function. The flow is: Application → getTotalEnergy() → Deluge Function Execution → Custom Logic → Function Completes → Successful Execution.

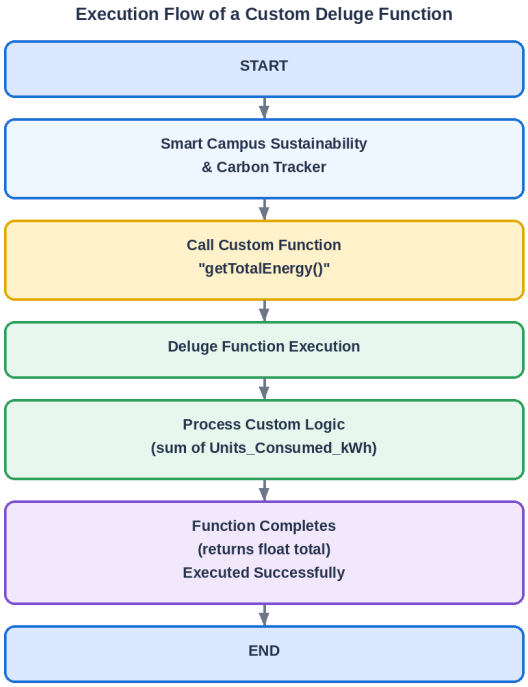


Figure 7: Execution Flow of a Custom Deluge Function (Supporting Diagram)

- **Call:** the application calls the custom function getTotalEnergy().
- **Execution:** the Deluge function is executed and its logic is processed. Here the logic is a single aggregate sum over the Energy_Consumption records.

- **Completion:** the function has the return type float, so it returns the total and completes with successful execution, as confirmed in Figure 5.

Low-Code and Pro-Code Extension

Low-code components provide the basic structure of an application, while pro-code extensions allow additional programmable logic and customisation. Figure 8 presents the two together as a combined approach.

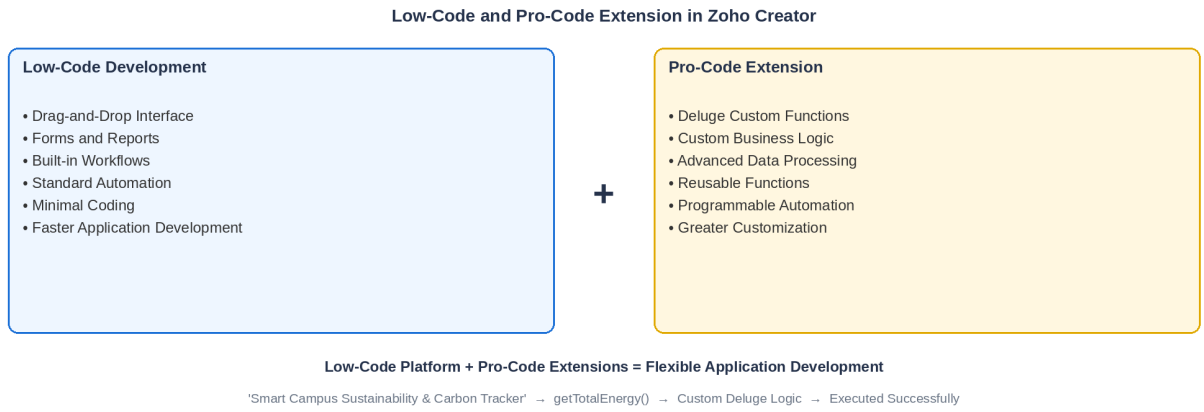


Figure 8: Low-Code and Pro-Code Extension in Zoho Creator (Supporting Diagram)

Low-Code	Pro-Code
Forms	Deluge functions
Reports	Custom logic
Workflows	Advanced processing
Standard automation	Reusable functions
Visual development	Programmable automation

The comparison describes the general roles of the two approaches. This experiment demonstrated only the creation and execution of a simple custom function; it did not connect the function to workflows, reports or other systems.

Result

The custom Deluge function getTotalEnergy was successfully created in Zoho Creator using the Default namespace, the float return type and no arguments. The function was saved, listed in the Functions section and executed, and the Execute window reported Executed Successfully.

Conclusion

Low-code platforms such as Zoho Creator simplify application development by providing forms, reports and workflows that can be assembled visually. Where specific requirements go beyond the standard components, pro-code extensions provide additional flexibility, and custom Deluge functions can introduce application-specific logic in a reusable form.

In this experiment, a custom function named getTotalEnergy was created with the Deluge language, the Default namespace and the float return type. The function was saved, listed in the Functions section and executed successfully. Although the function was simple, it illustrates the procedure for creating,

executing and verifying a custom function.

Combining low-code development with suitable pro-code extensions can support more flexible application development, in which standard functionality is built visually and specific logic, such as emission calculations and consumption rules, is added through code where it is required.